

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computacion

APLICACIONES WEB

INGENIERA DE SOFTWARE

GUERRERO ULLOA GLEISTON CICERON



1. Introducción

El desarrollo de aplicaciones web ha evolucionado significativamente en la última década, impulsado por la estandarización de tecnologías como HTML5, CSS3 y JavaScript ES6+. Estas herramientas permiten construir interfaces ricas, accesibles y responsivas sin depender de frameworks externos, lo que favorece la comprensión de los fundamentos del desarrollo frontend [1].

El presente informe documenta el desarrollo de una aplicación web denominada Explorador de Países, cuyo objetivo es demostrar el uso de tecnologías web estándar para consumir, procesar y visualizar datos provenientes de una API pública REST. La aplicación utiliza la API RestCountries v3.1, la cual proporciona información actualizada de más de 250 países del mundo de forma gratuita y sin necesidad de autenticación [2].

1.1 Objetivos

- Implementar una estructura HTML5 semántica y accesible siguiendo las recomendaciones del W3C.
- Aplicar CSS3 con Grid Layout y Flexbox para lograr un diseño responsivo adaptable a diferentes resoluciones de pantalla.
- Consumir una API pública REST mediante JavaScript ES6+ usando fetch y async/await.
- Organizar el código frontend bajo una arquitectura modular separada en capas (API, UI, utilidades, aplicación).
- Validar formularios HTML5 con JavaScript aplicando atributos nativos de validación y mensajes de error accesibles.

1.2 Justificación

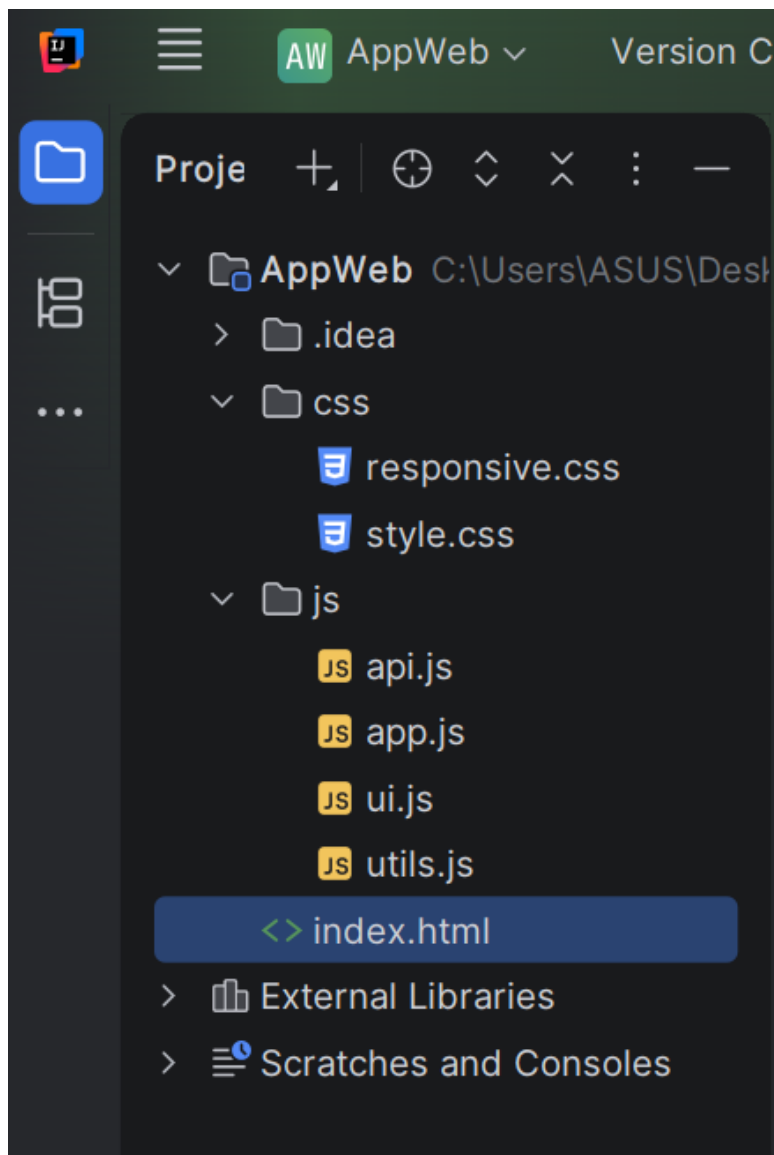
El dominio de las tecnologías web fundamentales es esencial para cualquier desarrollador frontend. Comprender cómo funciona el DOM, cómo se estructuran los datos en JSON y cómo interactúan los estilos con el layout permite construir soluciones más robustas y mantenibles. Este proyecto integra todos esos conocimientos en un caso de uso real: el consumo y visualización de datos geográficos a través de una API REST [3].

2. Desarrollo

2.1 Arquitectura del Proyecto

La aplicación sigue una arquitectura modular del frontend, donde cada archivo JavaScript tiene una responsabilidad única y bien definida. Esta separación de responsabilidades facilita el mantenimiento, la depuración y la escalabilidad del proyecto [4].

La estructura de directorios del proyecto es la siguiente:



2.2 HTML5 Semántico y Accesible

La estructura del documento HTML5 utiliza etiquetas semánticas que aportan significado al contenido, mejoran el posicionamiento SEO y facilitan la navegación asistida para

usuarios con discapacidades. Las etiquetas empleadas incluyen `<header>`, `<nav>`, `<section>`, `<main>`, `<article>` y `<footer>` [5].

Para garantizar la accesibilidad, se incorporaron atributos ARIA (Accessible Rich Internet Applications) en los elementos interactivos. Los atributos utilizados son:

- `aria-label`: describe el propósito de elementos de navegación y regiones.
- `aria-live`: notifica cambios dinámicos del contenido al lector de pantalla.
- `aria-describedby`: asocia mensajes de error con sus campos de formulario.
- `role`: define el rol semántico de elementos como alertas y regiones de estado.

El formulario de registro implementa 10 tipos de input distintos: text, email, password, tel, number, date, range, file, radio y checkbox, todos con validación nativa HTML5 mediante los atributos `required`, `pattern`, `min` y `max`.

2.3 CSS3 Responsivo con Grid y Flexbox

El diseño visual de la aplicación se implementó íntegramente con CSS3, sin el uso de frameworks externos. Se adoptó la metodología Mobile First, donde los estilos base corresponden a pantallas pequeñas y se amplían progresivamente mediante media queries [6].

Se utilizaron dos módulos complementarios de layout:

- Flexbox: empleado en el header, la barra de navegación, los controles de búsqueda y los botones del formulario. Permite la alineación y distribución de elementos en una sola dimensión.
- CSS Grid: empleado en el grid de tarjetas de países. Permite la disposición de elementos en dos dimensiones con columnas responsivas que se adaptan según el ancho de pantalla.

Los breakpoints definidos en `responsive.css` son:

- Base ($< 480\text{px}$): 1 columna en el grid.
- Tablet pequeña ($\geq 480\text{px}$): 2 columnas.
- Tablet ($\geq 768\text{px}$): 3 columnas.
- Desktop ($\geq 1024\text{px}$): 4 columnas.
- Desktop grande ($\geq 1280\text{px}$): 5 columnas.

Adicionalmente, se definieron variables CSS personalizadas (custom properties) para centralizar los valores de colores, tipografía y espaciados, facilitando la consistencia visual y el mantenimiento del código. Se implementaron animaciones con `@keyframes` para el spinner de carga y el efecto `fadeInUp` de las tarjetas.

2.4 JavaScript ES6+ y Arquitectura Modular

La lógica de la aplicación está distribuida en cuatro archivos JavaScript que siguen el principio de responsabilidad única (SRP) del diseño de software [4]:

`api.js` — Capa de acceso a datos: contiene exclusivamente las funciones de comunicación con la API REST. Utiliza `fetch` con `AbortController` para implementar un timeout de 8 segundos que evita esperas indefinidas en caso de fallo de red.

`utils.js` — Capa de utilidades: agrupa funciones auxiliares reutilizables como `formatearNumero()`, `obtenerNombre()`, `obtenerCapital()`, `obtenerBandera()`, `ordenarPorNombre()`, `toggleVisibilidad()` y `debounce()`. La función `debounce()` es especialmente importante ya que previene la ejecución excesiva del filtrado durante la escritura en el buscador.

`ui.js` — Capa de presentación: gestiona la manipulación del DOM, la creación dinámica de tarjetas con `createElement()` y `innerHTML`, y el control de los estados visuales de la aplicación (cargando, error, sin resultados).

`app.js` — Capa de aplicación: inicializa la aplicación, registra los event listeners y coordina la interacción entre las demás capas. Implementa además la lógica completa de validación del formulario con retroalimentación visual en tiempo real.

2.5 Consumo de API REST

RestCountries es una API REST pública y gratuita que expone información geográfica de todos los países del mundo. Su endpoint principal es <https://restcountries.com/v3.1/all>, el cual retorna un arreglo JSON con objetos que contienen datos como nombre, capital, región, población, área y bandera de cada país [2].

El proceso de consumo de la API sigue el siguiente flujo:

- La aplicación realiza una petición HTTP GET al endpoint de RestCountries.
- La respuesta JSON es parseada y almacenada en caché local (variable `paísesCache`).
- Los datos se filtran localmente según el texto de búsqueda y la región seleccionada.
- El módulo `ui.js` renderiza las tarjetas en el DOM a partir de los datos filtrados.

El uso de caché local elimina la necesidad de realizar peticiones adicionales a la API al filtrar, lo que mejora el rendimiento y reduce la latencia percibida por el usuario.

2.6 Resumen de Tecnologías Utilizadas

Tecnología	Tipo	Versión	Rol en el proyecto
HTML5	Lenguaje	5	Estructura semántica
CSS3	Estilos	3	Diseño responsivo
JavaScript	Lenguaje	ES6+	Lógica e interactividad
RestCountries API	API REST	v3.1	Fuente de datos

Tabla 1. Tecnologías empleadas en el desarrollo de la aplicación web.

2.7 Codigos de la pagina

Style.css

```
:root {  
  --color-primario: #1a237e;  
  --color-secundario: #283593;  
  --color-acento: #42a5f5;  
  --color-fondo: #f0f4f8;  
  --color-fondo-card: #ffffff;  
  --color-texto: #212121;  
  --color-texto-suave: #616161;
```

```
--color-borde: #e0e0e0;
--color-error: #c62828;
--color-error-fondo: #ffebee;

--fuente-principal: 'Segoe UI', Tahoma, Geneva,
Verdana, sans-serif;
--radio-borde: 12px;
--sombra-card: 0 2px 8px rgba(0, 0, 0, 0.1);
--sombra-hover: 0 6px 20px rgba(0, 0, 0, 0.15);
--transicion: all 0.25s ease;
}
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html {
  scroll-behavior: smooth;
}

body {
  font-family: var(--fuente-principal);
  background-color: var(--color-fondo);
  color: var(--color-texto);
  line-height: 1.6;
}

img {
```

```
    max-width: 100%;
    display: block;
}
a {
    color: var(--color-acento);
    text-decoration: none;
    transition: var(--transicion);
}
a:hover {
    text-decoration: underline;
}
.container {
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 1.5rem;
}
.site-header {
    background-color: var(--color-primario);
    color: #fff;
    padding: 1rem 0;
    position: sticky;
    top: 0;
    z-index: 100;
    box-shadow: 0 2px 8px rgba(0,0,0,0.3);
}
.site-header .container {
    display: flex;
    align-items: center;
```

```
    justify-content: space-between;
    flex-wrap: wrap;
    gap: 0.75rem;
}
.logo {
    font-size: 1.4rem;
    font-weight: 700;
    letter-spacing: 0.5px;
}
.nav {
    display: flex;
    gap: 1.5rem;
    align-items: center;
}
.nav a {
    color: #fff;
    font-weight: 500;
    opacity: 0.85;
    padding: 0.25rem 0;
    border-bottom: 2px solid transparent;
    transition: var(--transicion);
}
.nav a:hover {
    opacity: 1;
    border-bottom-color: var(--color-acento);
    text-decoration: none;
}
.hero {
```

```
    background: linear-gradient(135deg, var(--color-primario), var(--color-secundario));
    color: #fff;
    padding: 3.5rem 0 2.5rem;
    text-align: center;
}
.hero h2 {
    font-size: 2.2rem;
    font-weight: 800;
    margin-bottom: 0.75rem;
}
.hero p {
    font-size: 1.1rem;
    opacity: 0.9;
    margin-bottom: 2rem;
}
.controles {
    display: flex;
    gap: 1rem;
    justify-content: center;
    flex-wrap: wrap;
}
.buscador,
.filtro {
    padding: 0.75rem 1.1rem;
    border: none;
    border-radius: var(--radio-borde);
    font-size: 1rem;
```

```
    font-family: var(--fuente-principal);
    outline: none;
    transition: var(--transicion);
}

.buscador {
    width: 320px;
    max-width: 100%;
    background: rgba(255,255,255,0.95);
    color: var(--color-texto);
}

.buscador:focus {
    box-shadow: 0 0 0 3px var(--color-acento);
}

.filtro {
    background: rgba(255,255,255,0.95);
    color: var(--color-texto);
    cursor: pointer;
    min-width: 180px;
}

.filtro:focus {
    box-shadow: 0 0 0 3px var(--color-acento);
}

main {
    padding: 2rem 0 3rem;
}

.contador {
    color: var(--color-texto-suave);
    font-size: 0.95rem;
```

```
    margin-bottom: 1.25rem;
}
.card-pais {
    background: var(--color-fondo-card);
    border-radius: var(--radio-borde);
    box-shadow: var(--sombra-card);
    overflow: hidden;
    transition: var(--transicion);
    cursor: pointer;
    border: 1px solid var(--color-borde);
}
.card-pais:hover {
    transform: translateY(-5px);
    box-shadow: var(--sombra-hover);
}
.card-pais:focus-within {
    outline: 3px solid var(--color-acento);
}
.card-bandera {
    width: 100%;
    height: 160px;
    object-fit: cover;
}
.card-info {
    padding: 1.1rem 1.25rem 1.25rem;
}
.card-nombre {
    font-size: 1rem;
```

```
    font-weight: 700;
    margin-bottom: 0.6rem;
    color: var(--color-texto);
}

.card-detalle {
    font-size: 0.875rem;
    color: var(--color-texto-suave);
    line-height: 1.7;
}

.card-detalle span {
    color: var(--color-texto);
    font-weight: 600;
}

.loading {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 1rem;
    padding: 3rem 0;
    color: var(--color-texto-suave);
}

.spinner {
    width: 48px;
    height: 48px;
    border: 4px solid var(--color-borde);
    border-top-color: var(--color-primario);
    border-radius: 50%;
    animation: girar 0.8s linear infinite;
```

```
}  
@keyframes girar {  
  to { transform: rotate(360deg); }  
}  
.  
error {  
  background-color: var(--color-error-fondo);  
  border: 1px solid #ef9a9a;  
  border-radius: var(--radio-borde);  
  padding: 1.5rem;  
  text-align: center;  
  color: var(--color-error);  
}  
.  
btn-reintentar {  
  margin-top: 0.75rem;  
  padding: 0.5rem 1.25rem;  
  background-color: var(--color-error);  
  color: #fff;  
  border: none;  
  border-radius: 8px;  
  cursor: pointer;  
  font-size: 0.9rem;  
  transition: var(--transicion);  
}  
.  
btn-reintentar:hover {  
  background-color: #b71c1c;  
}  
.  
site-footer {  
  background-color: var(--color-primario);
```

```
    color: rgba(255,255,255,0.8);
    text-align: center;
    padding: 1.75rem 0;
    font-size: 0.9rem;
}

.site-footer a {
    color: var(--color-acento);
}

.footer-copy {
    margin-top: 0.4rem;
    opacity: 0.6;
    font-size: 0.8rem;
}

@keyframes fadeInUp {
    from {
        opacity: 0;
        transform: translateY(20px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.card-pais {
    animation: fadeInUp 0.4s ease both;
}

.seccion-formulario {
```

```
    background-color: #ffffff;
    padding: 3.5rem 0;
    border-top: 3px solid var(--color-acento);
}

.seccion-formulario h2 {
    font-size: 1.8rem;
    font-weight: 800;
    color: var(--color-primario);
    margin-bottom: 0.4rem;
}

.form-subtitulo {
    color: var(--color-texto-suave);
    margin-bottom: 2rem;
    font-size: 1rem;
}

.formulario {
    background: var(--color-fondo);
    border-radius: 16px;
    padding: 2rem;
    border: 1px solid var(--color-borde);
}

.form-fila {
    display: flex;
    gap: 1.25rem;
    flex-wrap: wrap;
}

.form-fila .form-grupo {
    flex: 1 1 240px;
```

```
}  
.form-grupo {  
  display: flex;  
  flex-direction: column;  
  gap: 0.35rem;  
  margin-bottom: 1.25rem;  
}  
.form-grupo label,  
.form-grupo legend {  
  font-size: 0.9rem;  
  font-weight: 600;  
  color: var(--color-texto);  
}  
.formulario input[type="text"],  
.formulario input[type="email"],  
.formulario input[type="password"],  
.formulario input[type="tel"],  
.formulario input[type="number"],  
.formulario input[type="date"],  
.formulario input[type="file"],  
.formulario select {  
  padding: 0.65rem 0.9rem;  
  border: 1.5px solid var(--color-borde);  
  border-radius: 8px;  
  font-size: 0.95rem;  
  font-family: var(--fuente-principal);  
  background-color: #fff;  
  color: var(--color-texto);
```

```
    transition: var(--transicion);
    outline: none;
}
.formulario input:focus,
.formulario select:focus {
    border-color: var(--color-acento);
    box-shadow: 0 0 0 3px rgba(66, 165, 245, 0.2);
}
.formulario input.invalido,
.formulario select.invalido {
    border-color: var(--color-error);
    box-shadow: 0 0 0 3px rgba(198, 40, 40, 0.15);
}
.formulario input.valido,
.formulario select.valido {
    border-color: #2e7d32;
    box-shadow: 0 0 0 3px rgba(46, 125, 50, 0.15);
}
.formulario input[type="range"] {
    padding: 0;
    border: none;
    background: transparent;
    accent-color: var(--color-primario);
    cursor: pointer;
    width: 100%;
}

.formulario input[type="file"] {
```

```
padding: 0.5rem;
cursor: pointer;
}
.fieldset-radio,
.fieldset-check {
border: 1.5px solid var(--color-borde);
border-radius: 8px;
padding: 0.75rem 1rem;
background: #fff;
}
.fieldset-radio legend,
.fieldset-check legend {
padding: 0 0.5rem;
font-size: 0.9rem;
font-weight: 600;
color: var(--color-texto);
}
.radio-grupo,
.check-grupo {
display: flex;
flex-wrap: wrap;
gap: 1rem;
margin-top: 0.5rem;
}
.radio-label,
.check-label {
display: flex;
align-items: center;
```

```
    gap: 0.4rem;
    font-size: 0.9rem;
    cursor: pointer;
    color: var(--color-texto);
}
.radio-label input,
.check-label input {
    accent-color: var(--color-primario);
    width: 16px;
    height: 16px;
    cursor: pointer;
}
.campo-error {
    font-size: 0.8rem;
    color: var(--color-error);
    display: none;
}
.campo-error.visible {
    display: block;
    animation: fadeInUp 0.2s ease;
}
.campo-ayuda {
    font-size: 0.8rem;
    color: var(--color-texto-suave);
}
.form-terminos {
    margin-top: 0.5rem;
}
```

```
.form-botones {
  display: flex;
  gap: 1rem;
  justify-content: flex-end;
  margin-top: 1.5rem;
  flex-wrap: wrap;
}

.btn-primario {
  padding: 0.7rem 2rem;
  background-color: var(--color-primario);
  color: #fff;
  border: none;
  border-radius: 8px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: var(--transicion);
}

.btn-primario:hover {
  background-color: var(--color-secundario);
  transform: translateY(-2px);
}

.btn-secundario {
  padding: 0.7rem 1.5rem;
  background-color: transparent;
  color: var(--color-texto-suave);
  border: 1.5px solid var(--color-borde);
  border-radius: 8px;
}
```

```
    font-size: 1rem;
    cursor: pointer;
    transition: var(--transicion);
}
.btn-secundario:hover {
    border-color: var(--color-texto-suave);
    color: var(--color-texto);
}
.form-exito {
    background-color: #e8f5e9;
    border: 1px solid #a5d6a7;
    color: #2e7d32;
    border-radius: 8px;
    padding: 1rem 1.25rem;
    text-align: center;
    font-weight: 600;
    margin-top: 1rem;
    animation: fadeInUp 0.3s ease;
}
.oculto {
    display: none !important;
}
```

responsive.css

```
.grid-paises {
    display: grid;
    grid-template-columns: 1fr;
    gap: 1.25rem;
```

```
}

.site-header .container {
  flex-direction: column;
  align-items: flex-start;
}

.nav {
  gap: 1rem;
}

.hero h2 {
  font-size: 1.6rem;
}

.hero {
  padding: 2.5rem 0 2rem;
}

.controles {
  flex-direction: column;
  align-items: center;
}

.buscador {
  width: 100%;
}

.filtro {
  width: 100%;
}

@media (min-width: 480px) {
  .grid-paises {
    grid-template-columns: repeat(2, 1fr);
    gap: 1.5rem;
  }

  .controles {
    flex-direction: row;
  }

  .buscador {
    width: 260px;
  }
}
```

```
}

.filtro {
  width: auto;
}
}

@media (min-width: 768px) {
  .grid-paises {
    grid-template-columns: repeat(3, 1fr);
    gap: 1.5rem;
  }

  .site-header .container {
    flex-direction: row;
    align-items: center;
  }

  .hero h2 {
    font-size: 2rem;
  }

  .hero {
    padding: 3rem 0 2.5rem;
  }

  .buscador {
    width: 300px;
  }
}

@media (min-width: 1024px) {
  .grid-paises {
    grid-template-columns: repeat(4, 1fr);
    gap: 1.75rem;
  }

  .hero h2 {
    font-size: 2.4rem;
  }

  .buscador {
    width: 340px;
  }
}
```



```

    }
  }

  @media (min-width: 1280px) {
    .grid-paises {
      grid-template-columns: repeat(5, 1fr);
      gap: 2rem;
    }
  }

  @media (prefers-reduced-motion: reduce) {
    *, *::before, *::after {
      animation-duration: 0.01ms !important;
      transition-duration: 0.01ms !important;
    }
  }
}

```

api.js

```

const API_BASE = 'https://restcountries.com/v3.1';
const CAMPOS =
  'fields=name,capital,region,population,flags,area';

async function obtenerTodosLosPaises() {
  const controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(),
8000);

  try {
    const respuesta = await
fetch(`${API_BASE}/all?${CAMPOS}`, {
      signal: controller.signal
    });
    clearTimeout(timeout);
    if (!respuesta.ok) throw new Error(`Error HTTP:
${respuesta.status}`);
    return await respuesta.json();
  } catch (error) {
    clearTimeout(timeout);
    throw error;
  }
}

```

```
}
```

app.js

```
const buscador =
document.getElementById('buscador');
const filtroRegion = document.getElementById('filtro-
region');
const btnReintentar = document.getElementById('btn-
reintentar');
let paisesCache = [];
async function iniciarApp() {
  mostrarLoading();
  try {
    paisesCache = await obtenerTodosLosPaises();
    ocultarLoading();
    renderizarPaises(paisesCache);
  } catch (error) {
    console.error('Error al cargar países:', error);
    mostrarError();
  }
}
function filtrarPaises() {
  const textoBusqueda =
buscador.value.trim().toLowerCase();
  const regionSeleccionada = filtroRegion.value;
  let resultado = [...paisesCache];
  if (regionSeleccionada) {
    resultado = resultado.filter(p => p.region ===
regionSeleccionada);
  }
  if (textoBusqueda.length >= 2) {
    resultado = resultado.filter(p =>
obtenerNombre(p).toLowerCase().includes(textoBusqueda)
);
  }
  renderizarPaises(resultado);
}
function marcarInvalido(campo, errorId) {
```

```
campo.classList.remove('valido');
campo.classList.add('invalido');
const errorEl = document.getElementById(errorId);
if (errorEl) errorEl.classList.add('visible');
}
function marcarValido(campo, errorId) {
    campo.classList.remove('invalido');
    campo.classList.add('valido');
    const errorEl = document.getElementById(errorId);
    if (errorEl) errorEl.classList.remove('visible');
}
function validarFormulario() {
    let esValido = true;
    const nombre = document.getElementById('nombre');
    if (!nombre.validity.valid ||
nombre.value.trim().length < 3) {
        marcarInvalido(nombre, 'nombre-error');
        esValido = false;
    } else {
        marcarValido(nombre, 'nombre-error');
    }
    const email = document.getElementById('email');
    if (!email.validity.valid) {
        marcarInvalido(email, 'email-error');
        esValido = false;
    } else {
        marcarValido(email, 'email-error');
    }
    const password =
document.getElementById('password');
    if (!password.validity.valid) {
        marcarInvalido(password, 'password-error');
        esValido = false;
    } else {
        marcarValido(password, 'password-error');
    }
    const telefono =
document.getElementById('telefono');
    if (telefono.value && !telefono.validity.valid) {
        marcarInvalido(telefono, 'telefono-error');
        esValido = false;
    } else {
```

```

        marcarValido(telefono, 'telefono-error');
    }
    const edad = document.getElementById('edad');
    if (!edad.validity.valid) {
        marcarInvalido(edad, 'edad-error');
        esValido = false;
    } else {
        marcarValido(edad, 'edad-error');
    }
    const fecha = document.getElementById('fecha-nacimiento');
    if (!fecha.value) {
        marcarInvalido(fecha, 'fecha-error');
        esValido = false;
    } else {
        marcarValido(fecha, 'fecha-error');
    }
    const regionPref = document.getElementById('region-preferida');
    if (!regionPref.value) {
        marcarInvalido(regionPref, 'region-error');
        esValido = false;
    } else {
        marcarValido(regionPref, 'region-error');
    }
    const terminos = document.getElementById('terminos');
    const terminosError = document.getElementById('terminos-error');
    if (!terminos.checked) {
        terminosError.classList.add('visible');
        esValido = false;
    } else {
        terminosError.classList.remove('visible');
    }
    return esValido;
}

function iniciarFormulario() {
    const form = document.getElementById('form-registro');
    if (!form) return;
    const rangeInput = document.getElementById('num-

```

```

favoritos');
    const valorRange = document.getElementById('valor-
range');
    rangeInput.addEventListener('input', () => {
        valorRange.textContent = rangeInput.value;
    });
    const camposTexto =
form.querySelectorAll('input[type="text"],
input[type="email"], input[type="password"],
input[type="tel"], input[type="number"],
input[type="date"], select');
    camposTexto.forEach(campo => {
        campo.addEventListener('blur', () => {
            if (campo.value) {
                campo.validity.valid
                    ? marcarValido(campo, `${campo.id}-
error`)
                    : marcarInvalido(campo,
`${campo.id}-error`);
            }
        });
    });
    form.addEventListener('submit', (e) => {
        e.preventDefault();
        if (validarFormulario()) {
            const exito = document.getElementById('form-
exito');
            toggleVisibilidad(exito, true);
            exito.scrollIntoView({ behavior: 'smooth',
block: 'center' });
            setTimeout(() => {
                form.reset();
                toggleVisibilidad(exito, false);
            }, 3000);
        }
    });
    form.querySelectorAll('.valido, .invalido').forEach(el
=> {
        el.classList.remove('valido',
'invalido');
    });
    valorRange.textContent = '5';
}, 3000);
}

```

```

    });
}
function mostrarLoading() {
    loadingEl.style.display = 'flex';
    loadingEl.classList.remove('oculto');
    toggleVisibilidad(errorEl, false);
    gridPaises.innerHTML = '';
    contadorEl.textContent = '';
}
function ocultarLoading() {
    toggleVisibilidad(loadingEl, false);
    loadingEl.style.display = 'none';
}
buscador.addEventListener('input',
debounce(filtrarPaises, 350));
filtroRegion.addEventListener('change', filtrarPaises);

btnReintentar.addEventListener('click', () => {
    buscador.value = '';
    filtroRegion.value = '';
    iniciarApp();
});
document.addEventListener('DOMContentLoaded', () => {
    iniciarApp();
    iniciarFormulario();
});

```

ui.js

```

const gridPaises = document.getElementById('grid-
paises');
const loadingEl = document.getElementById('loading');
const errorEl = document.getElementById('error');
const contadorEl = document.getElementById('contador');

function crearTarjetaPais(pais) {
    const nombre = obtenerNombre(pais);
    const capital = obtenerCapital(pais);
    const bandera = obtenerBandera(pais);
    const region = pais.region || 'N/D';
    const poblacion = formatearNumero(pais.population);
    const area = pais.area ?

```

```

` ${formatearNumero(Math.round(pais.area))} km²` : 'N/D';

const article = document.createElement('article');
article.classList.add('card-pais');
article.setAttribute('tabindex', '0');
article.setAttribute('aria-label', `País:
${nombre}`);

article.innerHTML = `

<div class="card-info">
  <h3 class="card-nombre">${nombre}</h3>
  <p class="card-detalle">
    <span>Capital:</span> ${capital}<br>
    <span>Región:</span> ${region}<br>
    <span>Población:</span> ${poblacion}<br>
    <span>Área:</span> ${area}
  </p>
</div>
`;
return article;
}

function renderizarPaises(paises) {
  gridPaises.innerHTML = '';

  if (paises.length === 0) {
    gridPaises.innerHTML = `
    <p style="grid-column:1/-1; text-align:center;
color:var(--color-texto-suave); padding:2rem 0;">
      No se encontraron países con ese criterio.
    </p>
  `;
    actualizarContador(0);
    return;
  }
}

```

```

ordenarPorNombre(paises).forEach(pais => {
    gridPaises.appendChild(crearTarjetaPais(pais));
});

actualizarContador(paises.length);
}

function actualizarContador(cantidad) {
    contadorEL.textContent = cantidad === 1
        ? `Mostrando 1 país`
        : `Mostrando ${formatearNumero(cantidad)}
países`;
}

function mostrarLoading() {
    loadingEL.style.display = 'flex';
    loadingEL.classList.remove('oculto');
    toggleVisibilidad(errorEL, false);
    gridPaises.innerHTML = '';
    contadorEL.textContent = '';
}

function ocultarLoading() {
    loadingEL.style.display = 'none';
    loadingEL.classList.add('oculto');
}

function mostrarError() {
    loadingEL.style.display = 'none';
    loadingEL.classList.add('oculto');
    toggleVisibilidad(errorEL, true);
    gridPaises.innerHTML = `
    <p style="grid-column:1/-1; text-align:center;
padding:3rem 0;
        color:var(--color-texto-suave); font-
size:1.1rem;">
        No hay países registrados por el momento.
    </p>
`;
}

```



```

function formatearNumero(numero) {
  if (!numero && numero !== 0) return 'N/D';
  return numero.toLocaleString('es-ES');
}

function obtenerNombre(pais) {
  return pais.name?.common || 'Desconocido';
}

function obtenerCapital(pais) {
  if (!pais.capital || pais.capital.length === 0) return 'Sin capital';
  return pais.capital[0];
}

function obtenerBandera(pais) {
  return pais.flags?.svg || pais.flags?.png || '';
}

function ordenarPorNombre(paises) {
  return [...paises].sort((a, b) =>
    obtenerNombre(a).localeCompare(obtenerNombre(b))
  );
}

function toggleVisibilidad(elemento, visible) {
  if (visible) {
    elemento.classList.remove('oculto');
  } else {
    elemento.classList.add('oculto');
  }
}

function debounce(funcion, espera = 400) {
  let temporizador;
  return function (...args) {
    clearTimeout(temporizador);
    temporizador = setTimeout(() => funcion.apply(this,
args), espera);
  };
}

```

Index.html

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-

```

```

scale=1.0" />
    <meta name="description" content="Explorador de países del
mundo usando RestCountries API" />
    <title>Explorador de Países</title>
    <link rel="stylesheet" href="css/style.css" />
    <link rel="stylesheet" href="css/responsive.css" />
</head>
<body>

<header class="site-header">
    <div class="container">
        <h1 class="logo"> Explorador de Países</h1>
        <nav class="nav" aria-label="Navegación principal">
            <a href="#inicio">Inicio</a>
            <a href="#países">Países</a>
            <a href="#registro">Registro</a>
            <a href="#acerca">Acerca</a>
        </nav>
    </div>
</header>

<section id="inicio" class="hero" aria-labelledby="hero-titulo">
    <div class="container">
        <h2 id="hero-titulo">Descubre el mundo</h2>
        <p>Explora información de más de 250 países: población,
capital, región y bandera.</p>
        <div class="controles">
            <input
                type="search"
                id="buscador"
                class="buscador"
                placeholder="Buscar país..."
                aria-label="Buscar país"
            />
            <select id="filtro-region" class="filtro" aria-
label="Filtrar por región">
                <option value="">Todas las regiones</option>
                <option value="Africa">África</option>
                <option value="Americas">América</option>
                <option value="Asia">Asia</option>
                <option value="Europe">Europa</option>
                <option value="Oceania">Oceanía</option>
            </select>
        </div>
    </div>
</section>

<main id="países" aria-label="Lista de países">
    <div class="container">

```

```

        <p class="contador" id="contador" aria-live="polite"></p>
        <div id="loading" class="loading" aria-live="polite"
role="status">
            <div class="spinner" aria-hidden="true"></div>
            <p>Cargando países...</p>
        </div>
        <div id="error" class="error oculto" role="alert">
            <p> No se pudo cargar la información. Intenta de
nuevo.</p>
            <button id="btn-reintentar" class="btn-
reintentar">Reintentar</button>
        </div>
        <section class="grid-paises" id="grid-paises" aria-
label="Tarjetas de países">
        </section>
    </div>
</main>

<section id="registro" class="seccion-formulario" aria-
labelledby="form-titulo">
    <div class="container">
        <h2 id="form-titulo">Registro de Usuario</h2>
        <p class="form-subtitulo">Crea tu cuenta para guardar tus
países favoritos</p>
        <form id="form-registro" class="formulario" novalidate>

            <div class="form-fila">
                <div class="form-grupo">
                    <label for="nombre">Nombre completo</label>
                    <input
                        type="text"
                        id="nombre"
                        name="nombre"
                        placeholder="Ej: Juan Pérez"
                        required
                        minlength="3"
                        maxlength="60"
                        pattern="[A-Za-záéíóúÁÉÍÓÚñÑ\s]+"
                        aria-describedby="nombre-error"
                        autocomplete="name"
                    />
                    <span class="campo-error" id="nombre-error"
role="alert">Solo letras y espacios, mínimo 3 caracteres.</span>
                </div>
                <div class="form-grupo">
                    <label for="email">Correo electrónico</label>
                    <input
                        type="email"
                        id="email"

```

```

        name="email"
        placeholder="usuario@correo.com"
        required
        aria-describedby="email-error"
        autocomplete="email"
    />
    <span class="campo-error" id="email-error"
role="alert">Ingresa un correo electrónico válido.</span>
    </div>
</div>

<div class="form-fila">
    <div class="form-grupo">
        <label for="password">Contraseña</label>
        <input
            type="password"
            id="password"
            name="password"
            placeholder="Mínimo 8 caracteres"
            required
            minlength="8"
            pattern="(?=.*\d)(?=.*[a-z])(?=.*[A-
Z]).{8,}"
            aria-describedby="password-error"
            autocomplete="new-password"
        />
        <span class="campo-error" id="password-error"
role="alert">Mínimo 8 caracteres, una mayúscula y un
número.</span>
    </div>
    <div class="form-grupo">
        <label for="telefono">Teléfono</label>
        <input
            type="tel"
            id="telefono"
            name="telefono"
            placeholder="0991234567"
            pattern="[0-9]{7,15}"
            aria-describedby="telefono-error"
            autocomplete="tel"
        />
        <span class="campo-error" id="telefono-error"
role="alert">Solo números, entre 7 y 15 dígitos.</span>
    </div>
</div>

<div class="form-fila">
    <div class="form-grupo">
        <label for="edad">Edad</label>

```

```

        <input
            type="number"
            id="edad"
            name="edad"
            placeholder="18"
            min="18"
            max="99"
            required
            aria-describedby="edad-error"
        />
        <span class="campo-error" id="edad-error"
role="alert">Debes tener entre 18 y 99 años.</span>
    </div>
    <div class="form-grupo">
        <label for="fecha-nacimiento">Fecha de
nacimiento</label>
        <input
            type="date"
            id="fecha-nacimiento"
            name="fecha_nacimiento"
            required
            aria-describedby="fecha-error"
        />
        <span class="campo-error" id="fecha-error"
role="alert">Selecciona tu fecha de nacimiento.</span>
    </div>
</div>

    <div class="form-fila">
        <div class="form-grupo">
            <label for="num-favoritos">Países favoritos a
mostrar: <strong id="valor-range">5</strong></label>
            <input
                type="range"
                id="num-favoritos"
                name="num_favoritos"
                min="1"
                max="20"
                value="5"
                aria-describedby="range-desc"
            />
            <span class="campo-ayuda" id="range-
desc">Selecciona cuántos países favoritos quieres ver (1-
20).</span>
        </div>
        <div class="form-grupo">
            <label for="foto-perfil">Foto de
perfil</label>
            <input

```

```

        type="file"
        id="foto-perfil"
        name="foto_perfil"
        accept="image/png, image/jpeg,
image/webp"
        aria-describedby="foto-ayuda"
    />
    <span class="campo-ayuda" id="foto-
ayuda">Formatos aceptados: PNG, JPG, WEBP.</span>
</div>
</div>

<div class="form-grupo">
    <fieldset class="fieldset-radio">
        <legend>Género</legend>
        <div class="radio-grupo">
            <label class="radio-label">
                <input type="radio" name="genero"
value="masculino" required />
                Masculino
            </label>
            <label class="radio-label">
                <input type="radio" name="genero"
value="femenino" />
                Femenino
            </label>
            <label class="radio-label">
                <input type="radio" name="genero"
value="otro" />
                Prefiero no decir
            </label>
        </div>
    </fieldset>
</div>

<div class="form-grupo">
    <fieldset class="fieldset-check">
        <legend>Intereses geográficos</legend>
        <div class="check-grupo">
            <label class="check-label">
                <input type="checkbox"
name="intereses" value="cultura" />
                Cultura
            </label>
            <label class="check-label">
                <input type="checkbox"
name="intereses" value="gastronomia" />
                Gastronomía
            </label>

```

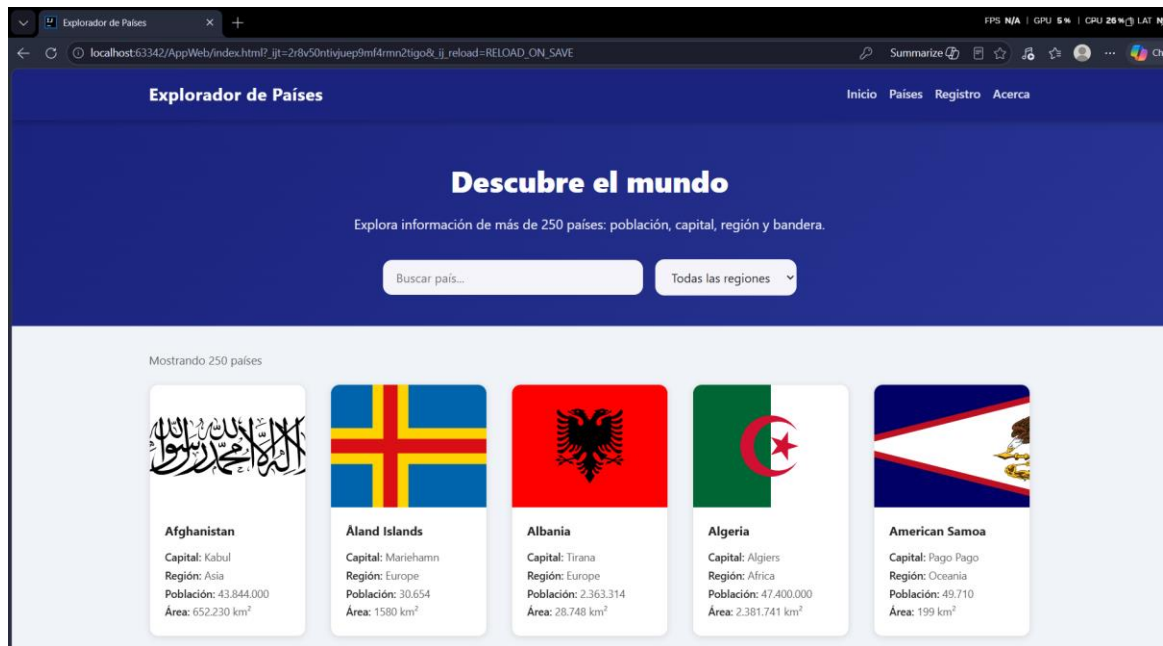
```

        <label class="check-label">
            <input type="checkbox"
name="intereses" value="historia" />
            Historia
        </label>
        <label class="check-label">
            <input type="checkbox"
name="intereses" value="naturaleza" />
            Naturaleza
        </label>
    </div>
</fieldset>
</div>

    <div class="form-grupo">
        <label for="region-preferida">Región
preferida</label>
        <select
            id="region-preferida"
            name="region_preferida"
            required
            aria-describedby="region-error"
        >
            <option value="">-- Selecciona una región --
</option>
            <option value="Africa">África</option>
            <option value="Americas">América</option>
            <option value="Asia">Asia</option>
            <option value="Europe">Europa</option>
            <option value="Oceania">Oceanía</option>
        </select>
        <span class="campo-error" id="region-error"
role="alert">Selecciona tu región preferida.</span>
    </div>

    <div class="form-grupo form-terminos">
        <label class="check-label">
            <input
                type="checkbox"
                id="terminos"
                name="terminos"
                required
                aria-describedby="terminos-error"
            />
            Acepto los <a href="#acerca">términos y
condiciones</a>
        </label>
        <span class="campo-error" id="terminos-error"
role="alert">Debes aceptar los términos para continuar.</span>
    </div>

```

The screenshot shows the registration form on the 'Explorador de Países' website. The form is divided into two columns. The left column contains fields for 'Nombre completo', 'Contraseña', 'Edad', 'Países favoritos a mostrar' (a slider set to 5), 'Género' (radio buttons for Masculino, Femenino, and Prefiero no decir), 'Intereses geográficos' (checkboxes for Cultura, Gastronomía, Historia, and Naturaleza), and 'Región preferida' (a dropdown menu). The right column contains fields for 'Correo electrónico', 'Teléfono', 'Fecha de nacimiento', and 'Foto de perfil' (a file upload button labeled 'Elegir archivo'). Below the 'Foto de perfil' field, it says 'Formatos aceptados: PNG, JPG, WEBP.' At the bottom of the form, there is a checkbox for 'Acepto los términos y condiciones'.

github: <https://github.com/XAML25/Tarea09-06-2026.git>

3. CONCLUSIONES

El desarrollo de la aplicación Explorador de Países permitió integrar de forma práctica los fundamentos del desarrollo web frontend mediante tecnologías estándar. A continuación se presentan las principales conclusiones obtenidas:

- La separación del código en módulos JavaScript independientes (api.js, utils.js, ui.js, app.js) demostró ser una práctica efectiva para mantener el código organizado, reutilizable y fácil de depurar. Esta arquitectura modular facilita la escalabilidad del proyecto a futuro.
- El uso de CSS Grid combinado con Flexbox y la metodología Mobile First permitió construir un diseño verdaderamente responsivo que se adapta correctamente desde pantallas de 320px hasta más de 1280px, sin necesidad de frameworks externos como Bootstrap.
- El consumo de la API REST de RestCountries mediante fetch y async/await demostró que JavaScript moderno dispone de herramientas nativas suficientes para manejar operaciones asíncronas de forma legible y eficiente.
- La implementación de HTML5 semántico con atributos ARIA no solo mejora la accesibilidad para usuarios con necesidades especiales, sino que también contribuye al SEO y a la calidad general del código.
- La validación nativa de formularios HTML5 complementada con JavaScript permitió ofrecer retroalimentación visual inmediata al usuario, mejorando significativamente la experiencia de uso.

4. Referencias

- [1] M. Pilgrim, HTML5: Up and Running. Sebastopol, CA: O'Reilly Media, 2010.
- [2] RestCountries, "REST Countries API v3.1 Documentation," restcountries.com. [En línea]. Disponible: <https://restcountries.com>. [Accedido: Jun. 2025].
- [3] N. C. Zakas, Professional JavaScript for Web Developers, 4th ed. Indianapolis, IN: John Wiley & Sons, 2019.
- [4] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ: Prentice Hall, 2008.
- [5] W3C, "HTML5: A vocabulary and associated APIs for HTML and XHTML," W3C Recommendation, Oct. 2014. [En línea]. Disponible: <https://www.w3.org/TR/html5>. [Accedido: Jun. 2025].
- [6] E. Marcotte, Responsive Web Design. New York, NY: A Book Apart, 2011.
- [7] MDN Web Docs, "CSS Grid Layout," Mozilla Developer Network. [En línea]. Disponible: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout. [Accedido: Jun. 2025].
- [8] MDN Web Docs, "Fetch API," Mozilla Developer Network. [En línea]. Disponible: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API. [Accedido: Jun. 2025].